

计算机程序设计基础(2)

--- C++程序设计(7)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2023.4.

1

5.9 动态束定与虚函数

- `virtual`所修饰的成员函数，被称为虚函数。
- 延迟到程序运行时进行的束定称为动态束定（也称为：动态绑定 `Dynamic Binding`）。
- 当一个类是从一个或多个类派生出来时，往往会出现多个函数名及修饰符相同且参数个数和参数类型也完全相同的成员函数，分别实现不同的功能。但这些函数不是重载函数（重载函数必须参数个数或参数类型不同，编译器可以静态束定(`Static Binding`)），编译器无法在编译时静态束定要调用哪一个成员函数，即使可以静态束定，往往也不一定符合编程者的真正意愿而导致结果错误。

2

- 虚函数延迟到程序运行时再进行动态束定，通过虚函数表(vtable)，由this指针决定调用哪一个同名的成员函数。
- 需要在重名的成员函数前加上 virtual 声明。
- 包含虚函数的类被称为多态类。
- 多态性（polymorphism）是面向对象的重要特征之一。
- 面向对象的另外两个重要特征是: 封装、继承。
- 提示：面向对象的三个重要特征是：
封装、继承、多态。

3

变为virtual函数之后，this指针通过当前类的虚函数表（vtable）调用本类的虚函数。

可以这么理解，过去的函数调用都是直接给函数的入口（起始）地址，属于静态束定，编译时就直接确定了。

而虚函数是间接调用，编译时只是给出此函数在虚函数表中的相对位置（同族的虚函数在每一个类的虚函数表中的相对位置相同），虚函数表中相应位置存放的是本类此虚函数的入口地址。所以这是动态束定，程序运行时由本类对象的this指针确定访问相应类的虚函数表，取得相应虚函数的入口地址再执行相应类的虚函数。

4

例如:

```
#include <iostream>
using namespace std;
class Shape { // 点类(X,Y)
public: Shape(double x=0, double y=0);
       double Area( ) const;
private: double X,Y;
};
Shape::Shape(double x, double y):X(x),Y(y){ }
double Shape::Area( ) const
{ return 0.0; }
class Circle : public Shape { // 圆类=点类+半径
public: Circle(double x, double y, double r);
       double Area( ) const;
private: double Radius;
};
```

5

```
Circle::Circle(double x, double y, double r)
: Shape(x,y), Radius(r) { } // Shape(x,y)调用基类构造函数
double Circle::Area( ) const
{ return 3.14159 * Radius * Radius; }
void fun(const Shape &sp) // 常引用对象调用的必须是常函数
{ cout << sp.Area( ) << endl; } // 去掉Area( )后的const将编译错误
void main( )
{ Circle C(2.0, 5.0, 3.65637);
  Shape S(3.0, 4.0);
  Shape *p[2]={&S, &C};
  fun(C); fun(S);
  for (int i=0; i<2; i++)
    cout << p[i]->Area( ) << endl;
}
```

运行结果:

0
0
0
0

类型兼容性原则: 基类指针可以指向它的派生类对象, 反之错误。

6

如果在相应函数名前加上virtual，把程序修改为：

```
#include <iostream>
using namespace std;
class Shape { // 点类(X,Y)
    public: Shape(double x, double y);
           virtual double Area( ) const; // 加上virtual
    private: double X,Y;
};
Shape::Shape(double x, double y):X(x),Y(y){ }
double Shape::Area( ) const
{ return 0.0; }
class Circle : public Shape { // 圆类=点类+半径
    public: Circle(double x, double y, double r);
           virtual double Area( ) const; // 加上virtual
    private: double Radius;
};
```

7

```
Circle::Circle(double x, double y, double r)
    : Shape(x,y), Radius(r)
{ }
double Circle::Area( ) const
{ return 3.14159 * Radius * Radius; }
void fun(const Shape &sp)
{ cout << sp.Area( ) << endl; }
void main( )
{   Circle C(2.0, 5.0, 3.65637);
    Shape S(3.0, 4.0);
    Shape *p[2]={&S, &C};
    fun(C); fun(S);
    for (int i=0; i<2; i++)
        cout << p[i]->Area( ) << endl;
}
```

运行结果：

42
0
0
42

8

5.9 动态绑定与虚函数（续1）

在派生类中重定义虚函数时，必须满足：

1. 与基类的虚函数参数个数相同；
2. 与基类的虚函数参数类型一一对应；
3. 函数返回类型或修饰词（如`const`）与基类的虚函数相同，或都返回指针或引用(可以不同类型)。

若满足上述三条件，有时可以省略`virtual`关键字，编译器会自动认为是同族虚函数。

但从程序可读性角度考虑，建议保留`virtual`关键字。

9

5.9.1 关于类型兼容性原则

网上一组关于类型兼容性的描述：

- （1）子类对象可以当作父类对象使用
- （2）子类对象可以直接赋值给父类对象
- （3）子类对象可以直接初始化父类对象
- （4）父类引用可以直接引用子类对象
- （5）父类指针可以直接指向子类对象

10

这里父类是指基类，子类是指派生类，所以类型兼容性原则可以改写为：

- (1) 派生类对象可以当作基类对象使用
- (2) 派生类对象可以直接赋值给基类对象
- (3) 派生类对象可以直接初始化基类对象
- (4) 基类引用可以直接引用派生类对象
- (5) 基类指针可以直接指向派生类对象

Base	Derived
------	---------

一个派生类对象中总是包含基类部分，在内存分布上，基类部分在前面，这样基类部分的起始地址与整个派生类的起始地址是一致的。因此：

- 基类指针可以直接指向派生类对象
- 基类引用可以直接引用派生类对象的基类部分

11

```
Rectangle::Rectangle(const Rectangle &p) : Shape(p)
```

是很典型的第（1）种，把派生类对象当作基类对象使用，让基类Shape的复制构造函数从派生类Rectangle的对象p中复制构造其基类Shape部分。

```
Derived d;
```

```
Base b;
```

```
b=d;
```

是第（2）种，直接赋值。把等号右边的派生类对象d，赋值给等号左边的基类对象b时，仅仅是把派生类中来自于基类的那一部分成员，赋值给等号左边的基类对象。

```
Derived d;
```

```
Base b(d);
```

是第（3）种，直接初始化。用派生类对象d直接给定义的基类对象b赋初值，当然仅仅是把派生类对象d中来自于基类的那一部分成员，复制构造赋值给基类对象b。

12

```
Derived d;
```

```
Base &b=d;
```

是第（4）种，引用赋值。把等号右边的派生象d，赋值给等号左边的基类b的引用，直接初始化了基类的引用。可以把派生类中的基类部分当做一个基类对象b来访问。

```
Derived d;
```

```
Base *b=&d;
```

是第（5）种，指针赋值。让基类的指针指向派生类对象，这个和上边的引用类似。b->只能访问基类部分的成员函数。

13

如何单独引用派生类对象中的基类部分，例如：

```
#include <iostream>
using namespace std;
```

```
class Base
{ public:
    Base(int x=0): ValB(x) { cout << ValB; }
    ~Base() { cout << ValB; }
    void print() { cout << ValB<<endl; }
private:
    int ValB;
};
```

```
class Derived : public Base
```

```
{ public:
    Derived(int x=0,int y=0): Base(x), ValD(y) { cout << ValD;}
    ~Derived() { cout << ValD; }
    void print() { Base::print(); cout << ValD<<endl; }
private:
    int ValD;
};
```

```
void main()
```

```
{Derived Obj1(2,3), Obj2(3,4);
```

```
Base &x=Obj1; // 基类引用可以直接引用派生类中继承的相应基类部分
x.print(); // =在这里是地址绑定,让x指向Obj1中的Base基类部分的首地址
Obj1.print(); Obj2.print();
}
```

运行结果：

```
23342
2
3
3
4
4332请按任意键继续...
```

14

```

#include <iostream> //引用派生类对象中的基类部分的综合例子
using namespace std;
class Base
{
public:
    Base(int x = 0) : ValB(x) { cout << ValB << " Base\n"; }
    Base(Base& p) : ValB(p.ValB) { cout << ValB << " Base Copy\n"; }
    ~Base() { cout << ValB; }
    void print() { cout << ValB << " Base print\n"; }
private:
    int ValB;
};
class Derived : public Base
{
public:
    Derived(int x=0, int y=0) : Base(x),ValD(y){ cout<<ValD<<" Derived\n"; }
    Derived(Derived& p) : Base(p),ValD(p.ValD){ cout << ValD<<" Derived Copy\n"; }
    ~Derived() { cout << ValD; }
    void print() { Base::print(); cout << ValD << " Derived print\n"; }
private:
    int ValD;
};

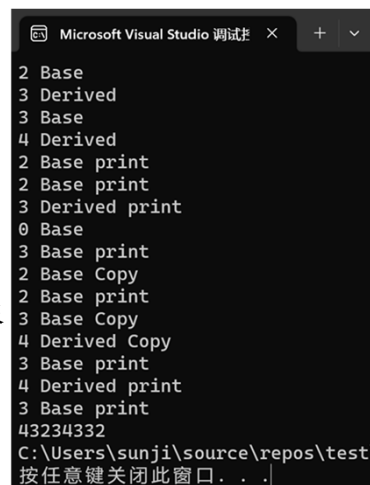
```

15

```

int main( )
{Derived Obj1(2,3), Obj2(3,4);
Base &x=Obj1; // 基类引用直接引用派生类中相应基类部分
x.print(); //在这里是地址绑定,让x指向Obj1中的Base基类部分的首地址
Obj1.print();
Base y;
y=Obj2; // 基类对象用派生类对象赋值
y.print();
Base z(Obj1);
    // 由派生类对象复制构造基类对象
z.print();
Derived Obj3(Obj2);
    // 由派生类对象复制构造派生类对象
Obj3.print();
Base *b=&Obj3; // 基类指针指向派生类对象
b->print();
return 0;
}
运行结果:

```



```

Microsoft Visual Studio 调试  ×  +  ▾
2 Base
3 Derived
3 Base
4 Derived
2 Base print
2 Base print
3 Derived print
0 Base
3 Base print
2 Base Copy
2 Base print
3 Base Copy
4 Derived Copy
3 Base print
4 Derived print
3 Base print
43234332
C:\Users\sunji\source\repos\test\
按任意键关闭此窗口. . .

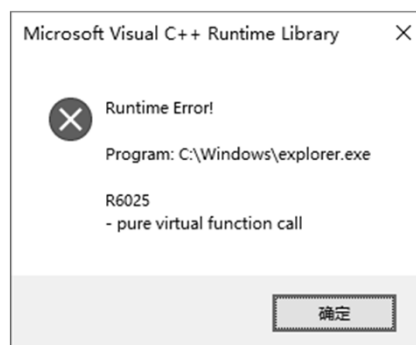
```

16

5.10 纯虚函数

- ◆ 纯虚函数是抽象类中的虚函数，在函数名后直接赋值为0，无函数体。
- ◆ 纯虚函数不同于空函数，是抽象的。
- ◆ 纯虚函数为抽象类中的成员函数。
- ◆ 包含纯虚函数的类被称为抽象类。
- ◆ 抽象类（无论类中是否有数据成员）无法实例化，因此不能用抽象类定义对象。
- ◆ 纯虚函数声明格式：
virtual 函数类型 函数名(参数表)=0;

17



这是我的计算机的Win10系统的文件资源管理器出现的一次运行错误：

R6025: pure virtual function call

也就是

R6025: 纯虚函数调用错误

18

```

#include <iostream>
#include <cmath>
using namespace std;
class Figure{    // Figure是一个抽象类
public:
    virtual double getArea( )=0;
}; // 纯虚函数，为整个类族提供了通用的外部接口语义
class Rectangle: public Figure {
public:
    Rectangle(double height=0,double width=0)
    { this->height=height;
      this->width=width;
    }
    double getArea( ) { return height*width; }
protected:
    double height, width;
};

```

19

```

class Triangle : public Figure {
public:
    Triangle(double a=0, double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    double getArea( )
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};

void main( )
{ Figure *Fig[ ]={new Rectangle(3,4), new Triangle(3,4,5)};
  // Figure基类指针可以指向其任一个派生对象
  cout << Fig[0]->getArea( ) << ' ' << Fig[1]->getArea( );
  delete Fig[0]; delete Fig[1];
}

```

这里可以定义Figure指针，但不能定义Figure对象，
因为抽象类Figure不能实例化。

20

运行结果:

12 6

请按任意键继续...

如果把主函数改为:

```
void main()  
{ Figure *Fig[ ] = {new Figure, new Rectangle(3,4), new Triangle(3,4,5)};  
  cout << Fig[0]->getArea() << ' ' << Fig[1]->getArea();  
  cout << ' ' << Fig[2]->getArea() << endl;  
  delete Fig[0]; delete Fig[1]; delete Fig[2];  
}
```

编译结果:

>test.cpp

1> test.cpp(28) : error C2259: "Figure": 不能实例化抽象类

1> 由于下列成员:

1> "double Figure::getArea(void)": 是抽象的

1> test.cpp(6) : 参见"Figure::getArea"的声明

1>test - 1 个错误, 0个警告

21

5.11 虚析构函数

```
#include <iostream>  
#include <cmath>  
using namespace std;  
class Figure{  
public:  
    virtual double getArea()=0;  
};  
class Rectangle: public Figure {  
public:  
    Rectangle(double height=0,double width=0)  
    { this->height=height; this->width=width; }  
    ~Rectangle() // 加入析构函数, 仅打印信息  
    { cout << "destruct Rectangle!\n"; }  
    double getArea()  
    { return height*width; }  
protected:  
    double height, width;  
};
```

22

```

class Triangle: public Figure {
public:
    Triangle(double a=0,double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    ~Triangle() // 加入析构函数，仅打印信息
    { cout << "destruct Triangle!\n"; }
    double getArea()
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};

void main()
{ Figure *Fig[ ] = {new Rectangle(3,4), new Triangle(3,4,5)};
  for (int i=0; i<2; i++)
      cout << Fig[i]->getArea() << endl;
  for (i=0; i<2; i++)
      delete Fig[i];
}

```

运行结果：

```

12
6
请按任意键继续...
未调用Rectangle和
Triangle的析构函数，因而没有正常
释放Rectangle和
Triangle对象的动态内存。

```

23

若在Figure类中加入： virtual ~Figure() { }

同时在相应析构函数名前加上virtual，其余不变：

```

#include <iostream>
#include <cmath>
using namespace std;
class Figure{
public:
    virtual double getArea()=0;
    virtual ~Figure() { } // 虚析构函数，但不能定义为纯虚函数
};
class Rectangle: public Figure {
public:
    Rectangle(double height=0,double width=0)
    { this->height=height; this->width=width; }
    virtual ~Rectangle() // 函数名前面可以加virtual，也可以不加
    { cout << "destruct Rectangle!\n"; }
    double getArea()
    { return height*width; }
protected:
    double height, width;
};

```

24

```

class Triangle: public Figure {
public:
    Triangle(double a=0,double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    virtual ~Triangle( ) //函数名前面可以加virtual, 也可以不加
    {cout << "destruct Triangle!\n"; }
    double getArea( )
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};

void main( )
{ Figure *Fig[ ] = {new Rectangle(3,4), new Triangle(3,4,5)};
  for (int i=0; i<2; i++)
      cout << Fig[i]->getArea( ) << endl;
  for (int i=0; i<2; i++)
      delete Fig[i];
}

```

运行结果:

```

12
6
destruct Rectangle!
destruct Triangle!
请按任意键继续...

```

25

- 在Figure类中加入: virtual ~Figure(){ }
- 将 ~Figure()
~Rectangle()
~Triangle()
- 变为同族的虚析构造函数。
- **注意:** 有虚析构造函数, 但没有对应的虚构造函数。
- 虚析构造函数使系统在执行过程中进行析构时, 根据this指针所指的当前对象, 动态决定使用相应对象的析构函数进行正确析构。
- 抽象类Figure中的虚析构函数~Figure()不能定义为纯虚函数, 因为派生类的析构函数~Rectangle()和~Triangle()会默认调用~Figure(), 而纯虚函数不能被调用只能被覆盖。
- 若不是虚析构造函数, 编译时将直接静态绑定, 将使得 delete Fig[i]都使用基类的~Figure(), 不能正确析构。

提示:

虚析构造函数是C++中唯一的函数名不同的虚函数族。

26

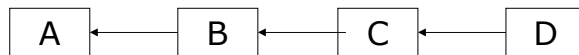
5.12 多继承

- ✓ 前面讨论的是单继承：一个派生类是从一个基类派生而来的。
- ✓ 若一个派生类是同时从两个和两个以上基类派生而来的，则称为多继承 (Multiple Inheritance, 多重继承)。

```
class D : public A, protected B, private C
{
    .....
}
```

- ✓ 多继承有几种情况：

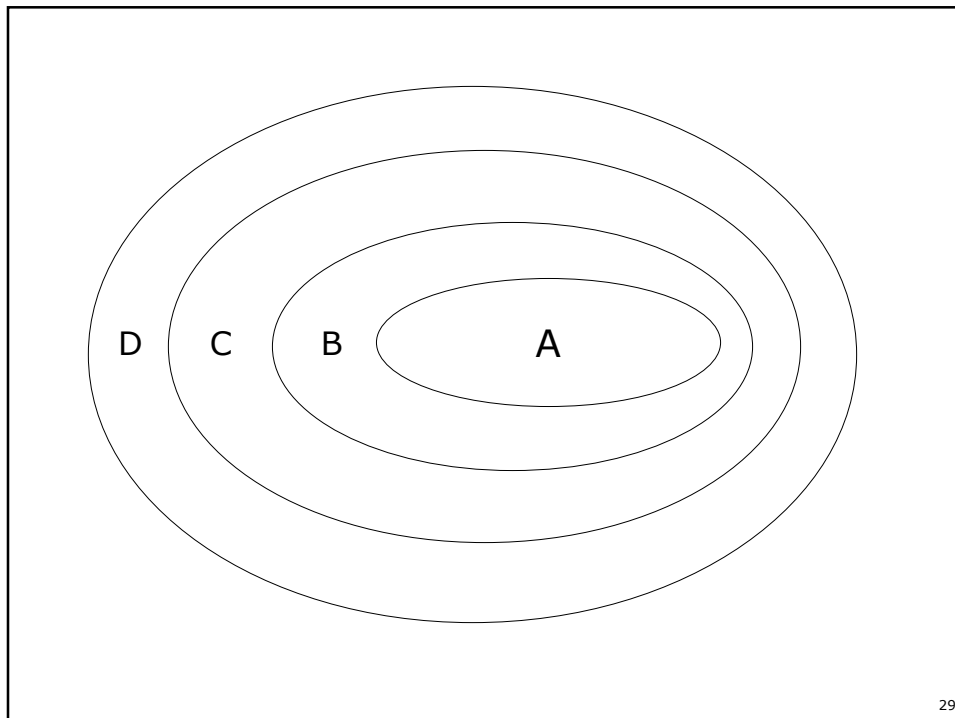
1. B从A派生，C从B派生，D从C派生，多层(重)派生



27

```
class A
{
    public:  A(...) {...}           // 构造函数
            A(const A &p){...} // 复制构造函数
    .....
};
class B : public A
{
    public:  B(...) : A(...) {...} // 构造函数
            B(const B &p) : A(p) {...} // 复制构造函数
    .....
};
class C : public B
{
    public:  C(...) : B(...) {...} // 构造函数
            C(const C &p) : B(p) {...} // 复制构造函数
    .....
};
class D : public C
{
    public:  D(...) : C(...) {...} // 构造函数
            D(const D &p) : C(p) {...} // 复制构造函数
    .....
};
```

28



29

注意：构造顺序：无论A、B、C类的成员是private还是protected，派生类的构造函数和复制构造函数只能使用其直接继承的基类的构造函数和复制构造函数构造派生类中的基类对象。

例如：D的构造函数只能使用C的构造函数，而不能直接使用B和A的构造函数构造；只能间接由C的构造函数使用B的构造函数，再间接由B的构造函数使用A的构造函数，从而依次构造出A、B、C基类对象，最终构造出D对象。

◆ 写成：

```
class D : public C
{
    public: D(...) : A(...),B(...),C(...) {...}    // 构造函数
           D(const D &p) : A(p),B(p),C(p) {...} // 复制构造函数
    .....
};
```

是错误的。

◆ **析构顺序：**派生类D的析构函数析构时，首先析构D，再调用其直接基类C的析构函数，C的析构函数先析构C，再调用其直接基类B的析构函数，B的析构函数先析构B，再调用其直接基类A的析构函数析构A，至此整个派生类D被析构完毕。

30

多层（重）派生示例：

```
#include <iostream>
using namespace std;
class A
{ public:
    A(int x) { a=x; cout << a << " A构造函数被调用\n"; }
    A(const A &p): a(p.a){ cout << a << " A复制构造函数被调用\n";}
    ~A() { cout << a << " A析构函数被调用\n"; }
protected:
    int a;
};
class B: public A
{ public:
    B(int a, int b) : A(a),b(b) { cout << b << " B构造函数被调用\n";}
    B(const B &p) : A(p),b(p.b)
    { cout <<b << " B复制构造函数被调用\n";}
    ~B() { cout << b << " B析构函数被调用\n"; }
protected:
    int b;
};
```

31

```
class C : public B
{ public:
    C(int a, int b,int c) : B(a,b), c(c)
    { cout << c << " C构造函数被调用\n";}
    C(const C &p) : B(p),c(p.c)
    { cout <<c << " C复制构造函数被调用\n";}
    ~C() { cout << c << " C析构函数被调用\n"; }
protected:
    int c;
};
class D : public C
{ public:
    D(int a, int b,int c,int d) : C(a,b,c),d(d)
    { cout << d << " D构造函数被调用\n";}
    D(const D &p) : C(p),d(p.d)
    { cout <<d << " D复制构造函数被调用\n";}
    ~D() { cout << d << " D析构函数被调用\n"; }
    void Display( );
protected:
    int d;
};
```

32


```

void D::Display( )
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; }
void main( )
{ D d(1,2,3,4);
  D d2(d);
  d.Display( );
  d2.Display( );
}

```

程序运行结果：

```

1 A构造函数被调用
2 B构造函数被调用
3 C构造函数被调用
4 D构造函数被调用
1 A复制构造函数被调用
2 B复制构造函数被调用
3 C复制构造函数被调用
4 D复制构造函数被调用
1 2 3 4
1 2 3 4
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
请按任意键继续...

```

33

如果把类D的构造函数和复制构造函数改为：

```

D(int a, int b,int c,int d) : a(a),b(b),c(c),d(d)
{ cout << d << " D构造函数被调用\n";}
D(const D &p) : a(p.a),b(p.b),c(p.c),d(p.d)
{ cout <<d << " D复制构造函数被调用\n";}

```

编译结果是：

```

1>c:\test.cpp(31) : error C2512: “C”: 没有合适的默认构造函数可用
1>c:\test.cpp(31) : error C2614: “D”: 非法的成员初始化:“c”不是基或成员
1>c:\test.cpp(31) : error C2614: “D”: 非法的成员初始化:“b”不是基或成员
1>c:\test.cpp(31) : error C2614: “D”: 非法的成员初始化:“a”不是基或成员
1>c:\test.cpp(32) : error C2512: “C”: 没有合适的默认构造函数可用
1>c:\test.cpp(32) : error C2614: “D”: 非法的成员初始化:“c”不是基或成员
1>c:\test.cpp(32) : error C2614: “D”: 非法的成员初始化:“b”不是基或成员
1>c:\test.cpp(32) : error C2614: “D”: 非法的成员初始化:“a”不是基或成员

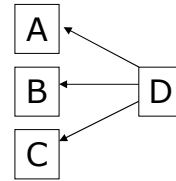
```

34

5.12 多继承 续1

2. 一个派生类同时从几个基类派生而来

```
class D : public A, protected B, private C
{
    .....
}
```

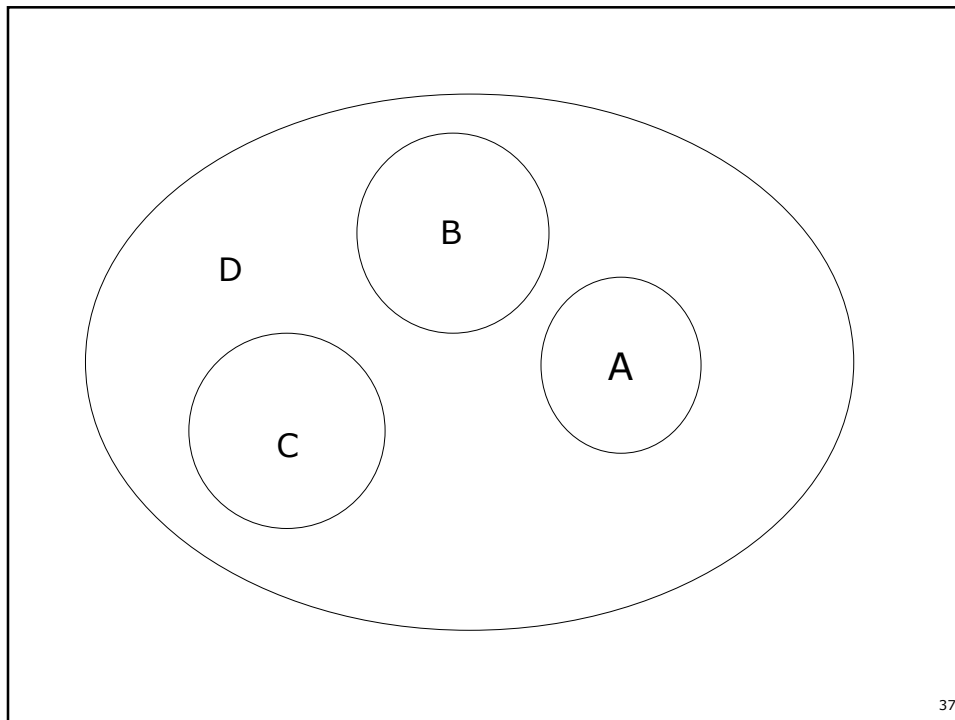


- ◆ 派生类将继承所有基类的全部成员和成员函数，并可以加入新的成员和成员函数。
- ◆ 当建立派生类的对象时，基类构造函数被依次执行，派生类的对象中所包含的基类子对象被基类的构造函数初始化。
- ◆ 但为了使用基类的某个特定的构造函数初始化派生类对象中所包含的基类子对象，必须在派生类的构造函数的成员初始化列表中显式表示对这个基类构造函数的调用。

35

```
class A
{
    public:    A(...) {...}    // 构造函数
              A(const A &p){...} // 复制构造函数
    .....
};
class B
{
    public:    B(...) {...}    // 构造函数
              B(const B &p){...} // 复制构造函数
    .....
};
class C
{
    public:    C(...): {...}    // 构造函数
              C(const C &p) {...} // 复制构造函数
    .....
};
class D : public A, public B, public C    // 多继承
{
    public:    D(...): A(...), B(...), C(...) {...} // 构造函数
              D(const D &p) : A(p),B(p),C(p){...} //复制构造函数
    .....
};
```

36



执行顺序：

- 派生类D的构造函数首先使用基类A、B、C的构造函数依次构造出A、B、C基类对象，再构造D的成员，最终构造出整个D对象。
- 派生类D的复制构造函数首先使用基类A、B、C的复制构造函数依次复制构造出A、B、C基类对象，再复制构造D的成员，最终复制构造出整个D对象。基类对象的构造顺序是由继承顺序决定的。
- 派生类D的析构函数析构时，首先析构D的成员，再依次调用基类C、B、A的析构函数，依次析构掉C、B、A的对象成员，至此整个D对象被析构完毕。基类对象的析构顺序是由继承顺序的逆序决定的。

38

同时从几个基类派生的示例：

```
#include <iostream>
using namespace std;
class A
{ public:
    A(int x) { a=x; cout << a << " A构造函数被调用\n"; }
    A(const A &p): a(p.a)
    { cout << a << " A复制构造函数被调用\n"; }
    ~A() { cout << a << " A析构函数被调用\n"; }
protected:
    int a;
};
class B
{ public:
    B(int b) : b(b) { cout << b << " B构造函数被调用\n"; }
    B(const B &p) : b(p.b)
    { cout << b << " B复制构造函数被调用\n"; }
    ~B() { cout << b << " B析构函数被调用\n"; }
protected:
    int b;
};
```

39

```
class C
{ public:
    C(int c) : c(c) { cout << c << " C构造函数被调用\n"; }
    C(const C &p) : c(p.c)
    { cout << c << " C复制构造函数被调用\n"; }
    ~C() { cout << c << " C析构函数被调用\n"; }
protected:
    int c;
};
class D : public A, public B, public C
{ public:
    D(int a, int b, int c, int d) : A(a), B(b), C(c), d(d)
    { cout << d << " D构造函数被调用\n"; }
    D(const D &p) : A(p), B(p), C(p), d(p.d)
    { cout << d << " D复制构造函数被调用\n"; }
    ~D() { cout << d << " D析构函数被调用\n"; }
    void Display() const ;
protected:
    int d;
};
```

40

```

void D::Display( ) const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; }
void main( )
{ D d(1,2,3,4);
  D d2(d);
  d.Display( );
  d2.Display( );
}

```

程序运行结果：

而且即使你把D的构造函数和复制构造函数的构造序列修改了顺序：

```

D(int a, int b,int c,int d) : C(c), B(b), A(a), d(d)
D(D &p) : C(p),B(p),A(p),d(p.d)

```

运行时仍按照A、B、C的顺序构造。因为基类的构造顺序是由继承顺序决定的：

```
class D : public A, public B, public C
```

```

1 A构造函数被调用
2 B构造函数被调用
3 C构造函数被调用
4 D构造函数被调用
1 A复制构造函数被调用
2 B复制构造函数被调用
3 C复制构造函数被调用
4 D复制构造函数被调用
1 2 3 4
1 2 3 4
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
请按任意键继续...

```

41

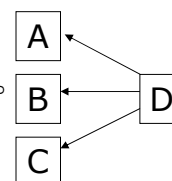
5.13 多继承的歧义性

1. 一个派生类同时从几个基类派生而来的，
如果几个基类中有同名的成员或成员函数。

```

class D : public A, public B, public C
{ ..... }

```



例如下面的例子中，几个基类中都有同名的保护成员a，因此D的成员函数Display()虽然可以直接访问每个基类的保护成员a，但如果写成：

```

void D::Display( ) const
{ cout << a<<" "<<a<<" "<<a<<" "<<b<<endl; }

```

将出现歧义性(ambiguous)，因为编译系统无法确定是要打印哪个基类的保护成员a，因而显示编译错误，需要在相应成员或成员函数前加基类名限定(像A::)。

42

```

#include <iostream>
using namespace std;
class A
{
    public:    A(int x) { a=x;}    // 构造函数
              A(const A &p): a(p.a){ } // 复制构造函数
    protected: int a;
};
class B
{
    public:    B (int x) : a(x) { } // 构造函数
              B(const B &p) : a(p.a){ } // 复制构造函数
    protected: int a;
};
class C
{
    public:    C (int x) { a=x;} // 构造函数
              C(const C &p) { a=p.a;} // 复制构造函数
    protected: int a;
};
class D : public A, public B, public C
{
    public:    D(int a, int b, int c, int d): A(a), B(b), C(c)
              { b=d; } // 构造函数
              D(const D &p) : A(p), B(p), C(p){b=p.b;} //复制构造函数
              void Display( ) const;
    protected: int b;
};

```

43

```

void D::Display( ) const
{ cout << a<<" "<<a<<" "<<a<<" "<<b<<endl; } // line 26
void main( )
{ D d(1,2,3,4);
  D d2(d);
  d.Display( );
  d2.Display( );
}

```

44

编译结果:

```
1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test - 3 个错误, 0个警告
```

45

而如果改为:

```
void D::Display( ) const
{ cout << A::a <<" "<< B::a <<" "<< C::a <<" "<<b<<endl; }
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
编译无任何错误! 运行结果:
```

```
1 2 3 -858993460
```

```
1 2 3 -858993460
```

请按任意键继续...

结果为什么是这样呢? 因为D的构造函数:

```
D(int a, int b, int c, int d): A(a), B(b), C(c) {b=d;}
```

中参数b与成员b同名, b=d; 没有将d的值赋给成员b, 应改为:

```
D(int a, int b, int c, int d): A(a), B(b), C(c) {this->b=d; }
```

修改后运行结果:

```
1 2 3 4
```

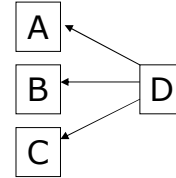
```
1 2 3 4
```

请按任意键继续...

46

2. 一个派生类同时从几个基类派生而来的，
如果派生类和几个基类中有同名的成员或成员函数。

```
class D : public A, public B, public C
{
    .....
}
```



例如下面的例子中，派生类和几个基类中都有同名的
保护成员a，因此D的成员函数Display()虽然可以直接
访问每个基类的保护成员a，但如果写成：

```
void D::Display() const
{
    cout << a << " " << a << " " << a << " " << a << endl;
}
```

47

```
#include <iostream>
using namespace std;
class A
{
    public:    A(int x) { a=x; } // 构造函数
             A(const A &p): a(p.a){ } // 复制构造函数
    protected: int a;
};
class B
{
    public:    B(int x) : a(x) { } // 构造函数
             B(const B &p) : a(p.a){ } // 复制构造函数
    protected: int a;
};
class C
{
    public:    C(int x) { a=x; } // 构造函数
             C(const C &p) { a=p.a; } // 复制构造函数
    protected: int a;
};
class D : public A, public B, public C
{
    public:    D(int a, int b, int c, int d): A(a), B(b), C(c)
             { this->a=d; } // 构造函数
             D(const D &p) : A(p), B(p), C(p){ a=p.a; } // 复制构造函数
    void Display() const;
    protected: int a;
};
```

48


```
void D::Display( ) const
{ cout << a<<" "<<a<<" "<<a<<" "<<a<<endl; } // line 26
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
编译无任何错误！运行结果：
```

4 4 4 4

4 4 4 4

请按任意键继续...

这是为什么呢？因为派生类D的同名成员a掩蔽了基类的同名成员，D::Display()直接看到的a是派生类D的a，如果想打印每个基类的成员a，必须在相应成员或成员函数前加基类名限定：

```
void D::Display( ) const
{ cout << A::a <<" "<< B::a <<" "<< C::a <<" "<<a<<endl; }
修改后运行结果：
```

1 2 3 4

1 2 3 4

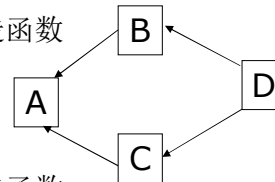
请按任意键继续...

49

3. 一个派生类同时从几个基类派生而来的，如果几个基类又是从同一个更老的共同基类派生而来。

例如：B，C都是从A派生来的，而D又是从B、C派生来的：

```
class A
{ .....};
class B : public A
{ public: B(...) : A(...) {...} // 构造函数
      B(const B &p) : A(p) {...} // 复制构造函数
  .....
};
class C : public A
{ public: C(...) : A(...) {...} // 构造函数
      C(const C &p) : A(p) {...} // 复制构造函数
  .....
};
class D : public B, public C
{ public: D(...) : B(...), C(...) {...} // 构造函数
      D(const D &p) : B(p), C(p) {...} // 复制构造函数
  .....
};
```



50

```

#include <iostream>
using namespace std;
class A
{
    public:    A(int x) { a=x;}    // 构造函数
              A(const A &p): a(p.a){ } // 复制构造函数
    protected: int a;
};
class B : public A
{
    public:    B (int x, int y) : A(x) {b=y; } // 构造函数
              B(const B &p) : A(p), b(p.b){ } // 复制构造函数
    protected: int b;
};
class C : public A
{
    public:    C (int x , int y) : A(x) {c=y; } // 构造函数
              C(const C &p) : A(p) {c=p.c;} // 复制构造函数
    protected: int c;
};
class D : public B, public C
{
    public:    D(int a, int b, int c, int d): B(a,b), C(a, c)
              { this->d=d;} // 构造函数
              D(const D &p) : B(p), C(p) {d=p.d;} // 复制构造函数
              void Display( ) const;
    protected: int d;
};

```

51

```

void D::Display( ) const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26
void main( )
{ D d(1,2,3,4); D d2(d); d.Display(); d2.Display(); }
编译结果:

```

test.cpp(26) : error C2385: 对“a”的访问不明确

1> 可能是“a”(位于基“A”中)

1> 也可能是“a”(位于基“A”中)

1>test - 1 个错误, 0个警告

出现错误的原因, 是编译器无法确定D::Display()要访问B和C中哪一个的a, 因为B和C中都有基类A, a是重复存在的。如果想打印其中一个基类的成员a, 必须在相应成员或成员函数前加基类名限定:

```

void D::Display( ) const
{ cout << B::a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26

```

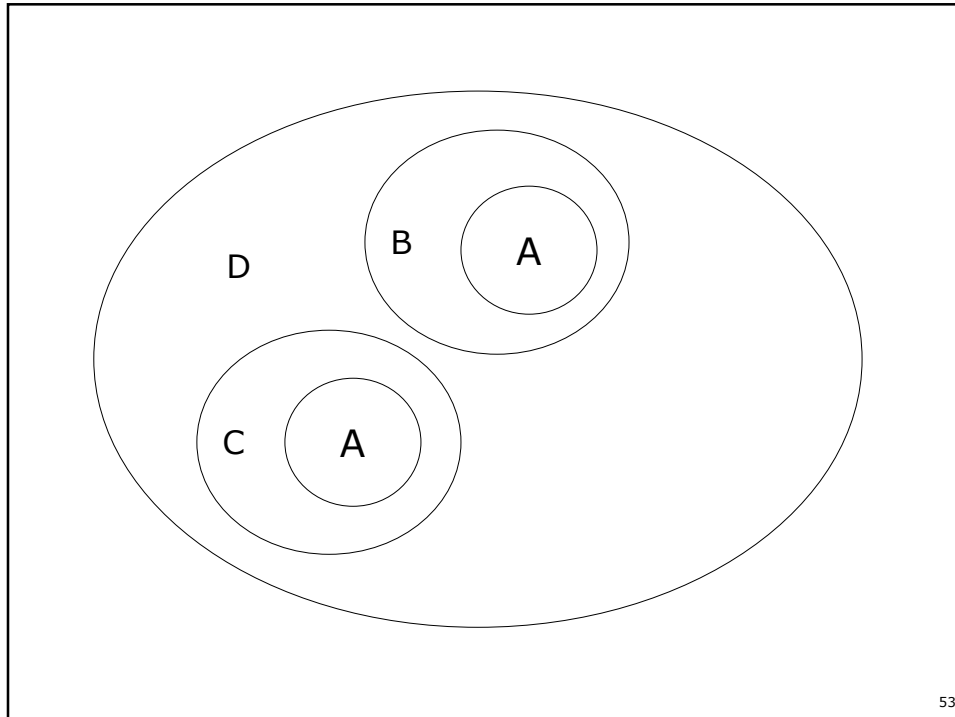
修改后编译没有错误, 运行结果:

1 2 3 4

1 2 3 4

请按任意键继续...

52



53

若把程序修改为：派生类D的两个直接基类中都有Display函数，而派生类中没有设置Display函数，程序代码为：

```
#include <iostream>
using namespace std;
class A
{
public:
    A(int x) { a=x; } // 构造函数
    A(const A &p): a(p.a) { } // 复制构造函数
protected: int a;
};
class B : public A
{
public:
    B(int x, int y) : A(x) { b=y; } // 构造函数
    B(const B &p) : A(p), b(p.b) { } // 复制构造函数
    void Display() const { cout << a << " " << b << endl; }
protected: int b;
};
class C : public A
{
public:
    C(int x, int y) : A(x) { c=y; } // 构造函数
    C(const C &p) : A(p) { c=p.c; } // 复制构造函数
    void Display() const { cout << a << " " << c << endl; }
protected: int c;
};
class D : public B, public C
{
public:
    D(int a, int b, int c, int d): B(a,b), C(a, c)
    { this->d=d; } // 构造函数
    D(const D &p) : B(p), C(p) { d=p.d; } // 复制构造函数
protected: int d;
};
```

54

```
void main( )
{ D d(1,2,3,4);   D d2(d);   d.Display( );
  d2.Display( );
}
```

编译结果:

1>test.cpp

1>test.cpp(27) : error C2385: 对 “Display” 的访问不明确

1> 可能是 “Display”(位于基 “B” 中)

1> 也可能是 “Display”(位于基 “C” 中)

1>test.cpp(27) : error C3861: “Display”: 找不到标识符

1>test.cpp(28) : error C2385: 对 “Display” 的访问不明确

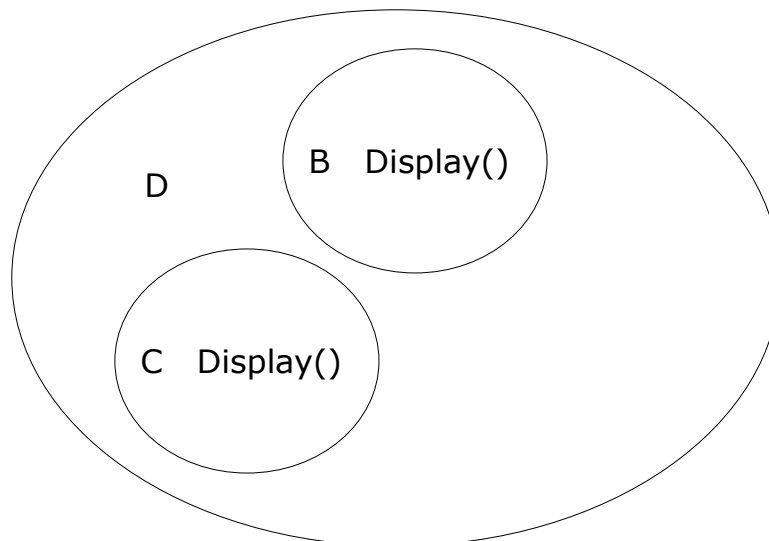
1> 可能是 “Display”(位于基 “B” 中)

1> 也可能是 “Display”(位于基 “C” 中)

1>test.cpp(28) : error C3861: “Display”: 找不到标识符

因为编译器无法确定 d.Display(); 和 d2.Display(); 中要使用的 Display 是基类 B 的还是基类 C 的, 因此出现编译错误。

55



56

如果加上基类名限定：

```
void main()  
{ D d(1,2,3,4);   D d2(d);   d.B::Display();  
  d2.C::Display();  
}
```

将不会出现编译错误，运行结果：

1 2

1 3

请按任意键继续...

其实派生类D中应该加一个Display函数，写为：

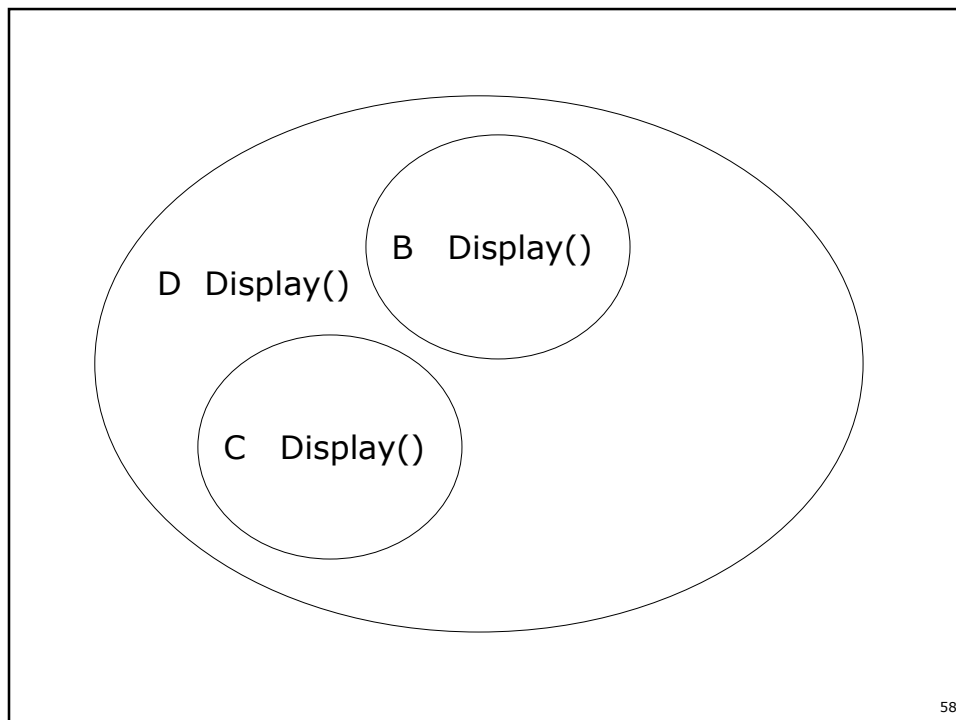
```
void Display() const { B::Display(); }
```

或 void Display() const { C::Display(); }

甚至 void Display() const { B::Display(); C::Display(); }

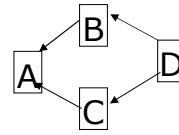
这样主程序中直接写：d.Display(); d2.Display(); 不会出现编译错误。

57



58

5.14 虚基类



- 一个派生类同时从几个基类派生而来的，如果几个基类又是从同一个更老的共同基类派生而来。
- 前面的例如：B，C都是从A派生来的，而D又是从B、C派生来的，这样D中将有2个A基类的副本，为了访问A基类的成员或成员函数，需要加限定名B::或C::。
- 虽然D中有2个A基类的副本有时候是必要的，但在许多情况下也是人们不希望出现的，因为多个基类的副本同时存在会造成占用更多内存，访问时不但麻烦还容易出错。
- 为了解决这个问题，C++引入了虚基类(virtual base class), 虚基类将使得派生类D在继承间接共同基类A时只保留一个A的副本。

59

```

#include <iostream>
using namespace std;
class A
{
    public: A(int x) { a=x; } // 构造函数
           A(const A &p): a(p.a){ } // 复制构造函数
    protected: int a;
};
class B : virtual public A //虚基类
{
    public: B(int x, int y) : A(x) {b=y; } // 构造函数
           B(const B &p) : A(p), b(p.b){ } // 复制构造函数
    protected: int b;
};
class C : virtual public A //虚基类
{
    public: C(int x , int y) : A(x) {c=y; } // 构造函数
           C(const C &p) : A(p) {c=p.c;} // 复制构造函数
    protected: int c;
};
class D : public B, public C
{
    public: D(int a, int b, int c, int d): B(a,b), C(a, c)
           {this->d=d;} // 构造函数 // line 20
           D(const D &p) : B(p), C(p){d=p.d;} //复制构造函数 //line 21
           void Display( ) const;
    protected: int d;
};
  
```

60

```
void D::Display() const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26
void main()
{ D d(1,2,3,4); D d2(d); d.Display(); d2.Display(); }
编译结果:
```

```
1>test.cpp(20) : error C2512: "A::A": 没有合适的默认构造函数可用
1>test.cpp(21) : error C2512: "A::A": 没有合适的默认构造函数可用
1>test - 2 个错误, 0 个警告
```

出现错误的原因是：当把B和C继承A的方式改为virtual继承方式后，A在最终的派生类D中只有一个副本，这样还间接通过B和C的构造函数和复制构造函数初始化基类A的成员，可能会因为B和C各自给不同的值初始化A，造成数据的不一致性。

因此，虚基类情况下，将由D的构造函数和复制构造函数直接调用间接基类A的构造函数和复制构造函数进行初始化，即使D的构造列表中没有给出A，系统也会按默认方式调用缺省值的A构造函数，但本程序中并没有缺省值的A构造函数，因此出现编译错误。程序应该改为：

61

```
#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
    A(const A &p): a(p.a){ } // 复制构造函数
    protected: int a;
};
class B : virtual public A //虚基类
{ public: B(int x, int y) : A(x) {b=y; } // 构造函数
    B(const B &p) : A(p), b(p.b){ } // 复制构造函数
    protected: int b;
};
class C : virtual public A //虚基类
{ public: C(int x, int y) : A(x) {c=y; } // 构造函数
    C(const C &p) : A(p) {c=p.c;} // 复制构造函数
    protected: int c;
};
class D : public B, public C
{ public: D(int a, int b, int c, int d): A(a), B(5, b), C(6, c)
    { this->d=d; } // 构造函数 // line 20
    D(const D &p) : A(p), B(p), C(p) {d=p.d;} // 复制构造函数
    void Display() const;
    protected: int d;
};
```

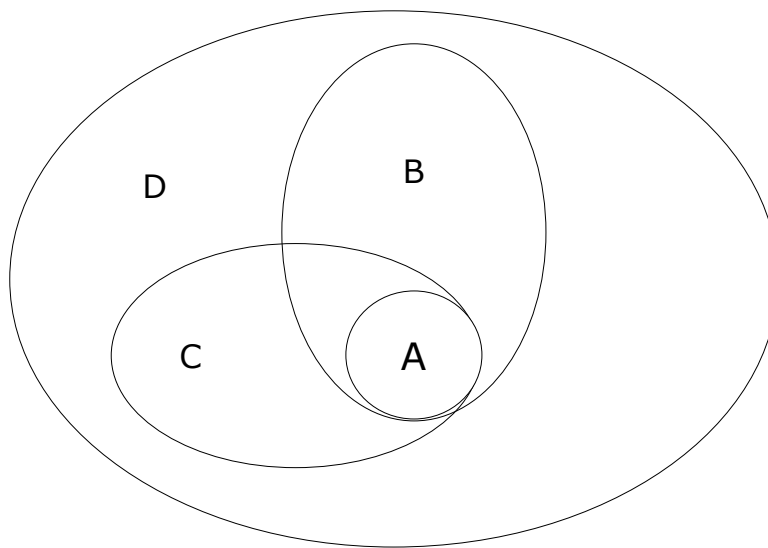
62

```
void D::Display() const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; }
void main()
{ D d(1,2,3,4); D d2(d); d.Display(); d2.Display(); }
修改后编译没有错误！运行结果：
1 2 3 4
1 2 3 4
请按任意键继续...
```

注意：1. 虽然D的构造函数和复制构造函数直接调用A的构造函数和复制构造函数进行初始化，但在书写B和C的构造函数和复制构造函数时，仍需调用A的构造函数和复制构造函数进行初始化，因为B和C很可能不止是D使用，B和C可以单独直接实例化生成对象，此时B和C的构造函数和复制构造函数将直接通过调用A的构造函数和复制构造函数来生成基类A的成员a并初始化。

2. 在生成D的对象d时，其中基类A的成员a只有一个副本，说明A的构造函数和复制构造函数只会被使用过一次，而不是3次，这个是通过virtual实现的。因为程序中故意给的B(5, b), C(6, c) 的值5、6并没有赋给a，说明此时B和C的构造函数并没有真去调用基类A的构造函数。

63



64

第8次作业：

见网络学堂第8次作业

2题必做，1题选做

65